Army Institute of Technology



Department of Information Technology (LAB Manual)

SY BTech

BIT25431A0B – Data Structures and Algorithms Lab

Staff In-charge
Prof. Kavita Arakeri
Prof.Pooja Adsul

Teaching Scheme Practical Session: 2 hrs/week	Examination Scheme Practical: 25 Marks Term Work: 25 Marks Credits: 1

LABORATORY OVERVIEW

Prerequisites:

1. Programming Fundamentals / Introduction to Programming.

Course Objectives :

1. To develop foundational knowledge of searching and sorting techniques used in computational problem solving.
2. To provide an understanding of linear data structures and their use in the implementation of stacks and queues.
3. To develop a non-linear Tree data structures and their associated operations.
4. To demonstrate a graph concepts and algorithms in solving graph-related problems.
5. To present hashing concepts and methods for efficient data storage and retrieval.

Course Outcomes:

On completion of the course, learner will be able to–

CO1: Apply searching and sorting techniques to solve computational problems.

CO2: Implement appropriate data structures for the realization of stacks and queues.

CO3: Implement operations on non-linear data structures such as trees.

CO4: Apply fundamental graph concepts and algorithms to solve graph-based problems.

CO5: Design appropriate hashing functions for efficient data storage and retrieval.

DEPARTMENT OF INFORMATION TECHNOLOGY

List of Assignments

Subject: Data Structure And Algorithm SE IT : A/B

SEM: II

No.	ASSIGNMENT NAME
1	Design a program to maintain a student database that performs the following tasks: 1. Add and store student details (ID, Name, CGPA) using dynamically allocated memory. 2. Implement Linear Search and Binary Search to find student records by ID. 3. Implement at least two Sorting Algorithms (Bubble Sort, Selection Sort, or Insertion Sort) to sort student records by: Name (Alphabetically), CGPA (Ascending/Descending) 4. Analyze and compare the performance of search operations before and after sorting
2	Sort Online Orders by Delivery Time using Merge Sort. Problem Statement: You are given a list of online orders, each with an estimated delivery time in minutes. Write a program to sort these orders using the Merge Sort algorithm so the delivery system can prioritize quicker deliveries first.
3	Implement stack as an abstract data type using singly linked list and use this ADT for conversion of infix expression to postfix, prefix
4	Implement Circular Queue using Array. Perform following operations on it. a) Insertion (Enqueue) b) Deletion (Dequeue) c) Display (Note: Handle queue full condition by considering a fixed size of a queue.)
5	Implement binary search tree and perform following operations: a) Insert (Handle insertion of duplicate entry) b) Delete c) Search d) Display tree (Traversal) e) Display - Depth of tree f) Display - Mirror image g) Create a copy h) Display all parent nodes with their child nodes i) Display leaf nodes j) Display tree level wise
6	Design and implement the Heap Sort algorithm to efficiently sort an array of integers in ascending order. The implementation should be optimized for time and space complexity and should clearly demonstrate the working principles of heap data structures (min-heap or max heap as applicable)
7	Represent a graph of your college campus using adjacency list /adjacency matrix. Nodes should represent the various departments/institutes and links should represent the distance between them. Find minimum spanning tree Using Prims & Kruskal's algorithm.
8	Implement a simple telephone directory using a hash table data structure. The goal is to quickly insert, search for, and delete client telephone numbers.
9	Implement File Handling Operations: Create, Read, Write and Delete

Assignment No. 1

AIM:

Design a program to maintain a student database that performs the following tasks: 1. Add and store student details (ID, Name, CGPA) using dynamically allocated memory. 2. Implement Linear Search and Binary Search to find student records by ID. 3. Implement at least two Sorting Algorithms (Bubble Sort, Selection Sort, or Insertion Sort) to sort student records by: Name (Alphabetically), CGPA (Ascending/Descending) 4. Analyze and compare the performance of search operations before and after sorting

OBJECTIVE

1. To study data structures and their implementations and applications
2. To understand different sorting and searching techniques

OUTCOME:

1. Understand linear data structure
2. Apply different sorting techniques on array of structure (Bubble, Insertion and Selection)
3. Apply different searching techniques on array of structure (Linear Search and Binary Search)
4. To perform basic analysis of algorithms with respect to time and space complexity.

THEORY:

1.**Structure**: Structure is collection of heterogeneous types of data. In C++, a structure collects different data items in such a way that they can be referenced as a single unit. Data items in a structure are called fields or members of the structure.

- Creating a Structure:

```
struct student
{
        int roll_no;
        string name;
        float sgpa;
}
```

- Declaring Structure Variables:

  After declaring a structure, you can define the structure variables. For instance, the following structure variables are defined with the structure data type of automobile from the previous section:

```
struct student s1, s2, s3;
```

Here three structure variables, s1, s2, s3 are defined by the structure of student. All three structure variables contain the three members of the structure student.

- Referencing Structure Members with the Dot Operator:

  Given the structure student and s1 is variable of type struct student we can access the fields in following way

```
s1.roll_no
s1.name
s1.sgpa
```

  Here the structure name and its member's name are separated by the dot (.) operator.

- Pointer to Structure:

  We can declare a pointer variable which is capable of storing the address of a structure variable as follows:

```
struct student *ptr;
```

- Referencing a Structure Member with -> (arrow operator)

  Using ptr we can access the fields of s1 as follows:

```
ptr->name
```

  Because of its clearness, the -> operator is more frequently used in programs than the dot operator

2. **Sorting Methods**:

Sorting is the process of arranging or ordering information in the ascending or descending order of the key values.

2.1 Bubble Sort: This is one of the simplest and most popular sorting methods. The basic idea is to pass through the array sequentially several times. In each pass we compare successive pairs of elements (A[i] with A[i+1]) and interchange the two if they are not in the required order. One element is placed in its correct position in each pass. In the first pass, the largest element will sink to the bottom, second largest in the second pass and so on. Thus, a total of n-1 passes are required to sort 'n' keys.

Analysis:
In this sort, n-1 comparisons takes place in 1st pass, n-2 in 2nd pass, n-3 in 3rd pass,
and so on. Therefore, total number of comparisons will be

```
(n-1) + (n-2) + (n-3) + ..... + 3 + 2 +1
```

This is an Arithmetic series in decreasing order

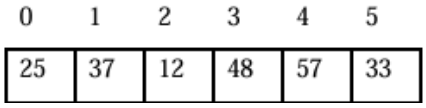
$$\text{Sum} = n/2\,[2a + (n-1)d]$$

Where, n = no. of elements in series,
a = first element in the series
d = difference between second element and first element.

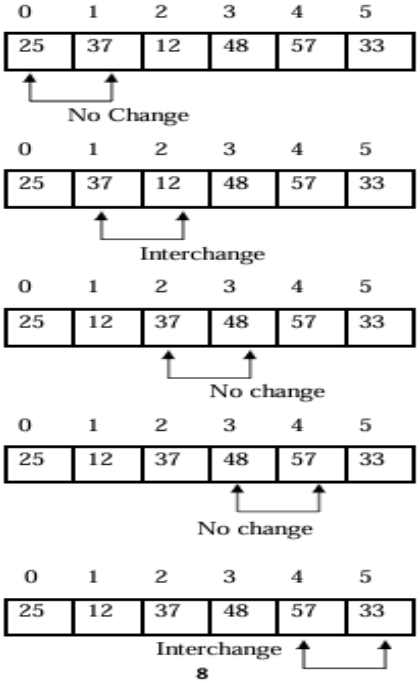
$$\text{Sum} = (n-1)/2\,[\,2 * 1 + ((n-1) - 1) * 1\,]$$
$$= n(n-1)/2$$

which is of order n2 i.e O(n2)

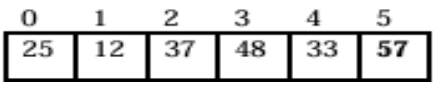
Example : (Bubble sort)



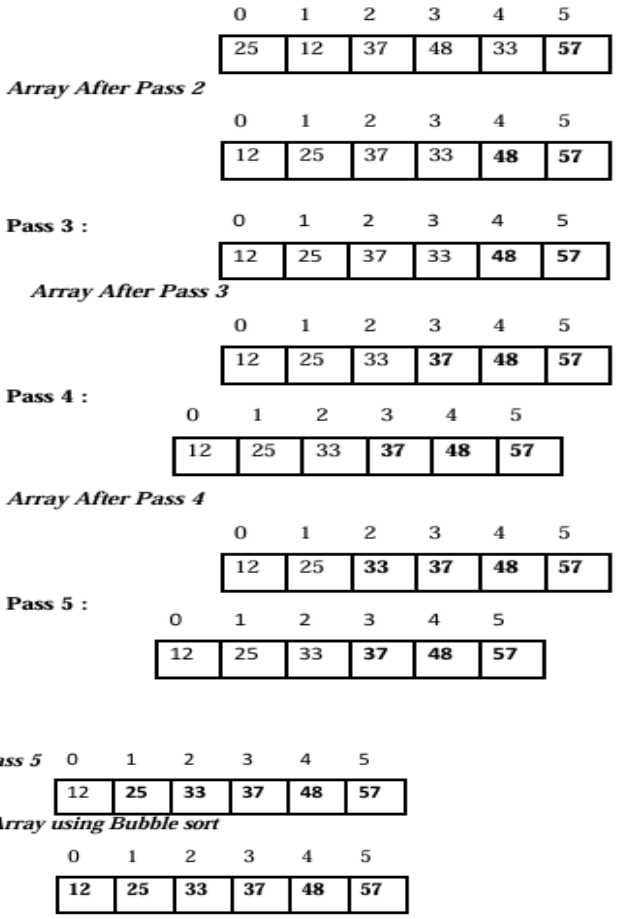
Pass 1: Compare two consecutive elements , if first element is greater than second then interchange the elements else no change

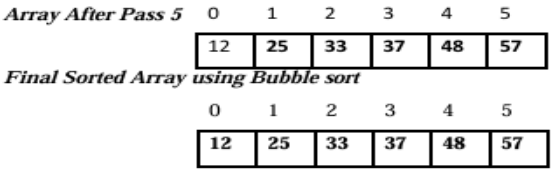


Array after Pass 1 :



Pass 2: Now we will show all the comparisons with the final array





2.2 Insertion Sort

Insertion sort is the sorting mechanism where the sorted array is built having one item at a time. The array elements are compared with each other sequentially and then arranged simultaneously in some particular order. The analogy can be understood from the style we arrange a deck of cards. This sort works on the principle of inserting an element at a particular position, hence the name Insertion Sort.

- The first step involves the comparison of the element with its adjacent element.
- And if at every comparison reveals that the element can be inserted at a particular position, then space is created for it by shifting the other elements one position to the right and inserting the element at the suitable position.
- The above procedure is repeated until all the element in the array is at their apt position.

Analysis:

- Worst Case Complexity: $O(n^2)$

Suppose, an array is in ascending order, and you want to sort it in descending order. In this case, worst case complexity occurs. Each element has to be compared with each of the other elements so, for every nth element, (n-1) number of comparisons are made.
Thus, the total number of comparisons = $n*(n-1) \sim n^2$

- Best Case Complexity: O(n)

When the array is already sorted, the outer loop runs for n number of times whereas the inner loop does not run at all. So, there are only n number comparisons. Thus, complexity is linear.

- Average Case Complexity: $O(n^2)$

It occurs when the elements of an array are in jumbled order (neither ascending nor descending).

Example:



2.3 Selection Sort

Selection sort is a simple sorting algorithm. This sorting algorithm, like insertion sort, is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end. Initially, the sorted part is empty and the unsorted part is the entire list.

This type of sorting is called Selection Sort as it works by repeatedly sorting elements. That is:

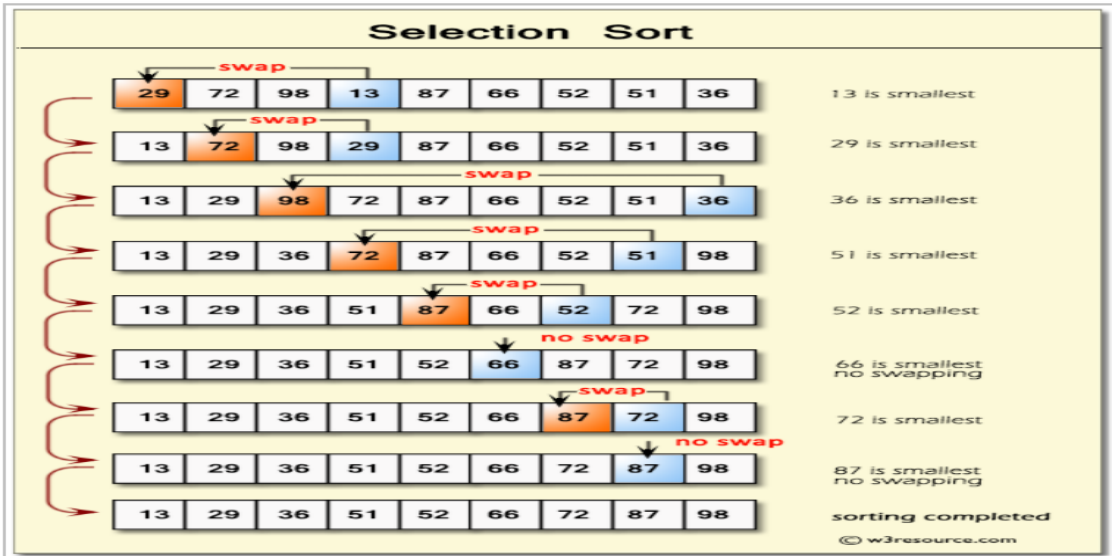
1. we first find the smallest value in the array and exchange it with the element in the first position,
2. Then find the second smallest element and exchange it with the element in the second position, and we continue the process in this way until the entire array is sorted.

Analysis

For each i from 1 to n - 1, there is one exchange and n - i comparisons, so there is a total of n - 1 exchanges and (n 1) + (n 2) + ...+2 + 1 = n(n 1)/2 comparisons.These observations hold, no matter what the input data is.

- Worst Case Complexity: $O(n^2)$
- Average Case Complexity: $O(n^2)$

It implies that the running time of Selection sort is quite insensitive to the input.
Example:



Comparition of sorting Techniques:

Algorithm	Time Complexity (Best)	Time Complexity (Average)	Time Complexity (Worst)
Bubble Sort	O(n)	O(n^2)	O(n^2)
Insertion Sort	O(n)	O(n^2)	O(n^2)
Selection Sort	O(n^2)	O(n^2)	O(n^2)

Algorithm for Bubble Sort

1. Start
2. For Pass = 1 to n - 1
3. For i = 0 to (n - Pass - 1)
4. If A[i].rollno > A[i+1].rollno
5. Swap A[i] and A[i+1]
6. End If
7. End For
8. End For
9. Stop

Algorithm for Insertion Sort

Algorithm Insertion _Sort (S,n) Where S is an structure array , n is the total number of records to sort
Start

1. For i = 1 to n - 1
2. X = S[i]
3. For j = i - 1 down to 0
4. If strcmp(X.Name, S[j].Name) < 0

5. S[j + 1] = S[j]
6. Else, break
7. End For
8. S[j + 1] = X
9. End For
10. Stop

Algorithm Quicksort
Quicksort (Struct ST [], int s, int l) where ST is the array of Structure, s is the index of starting element and l is the index of last element.

1. Start
2. Select ST[s] as the pivot element
3. Set b = s + 1
4. Set e = l
5. While b ≤ e
6. Increment b till ST[b].SGPA ≤ pivot element
7. Decrement e till ST[e].SGPA > pivot element
8. If b < e
9. Swap(ST[b], ST[e])
10. End if
11. End while
12. Swap(ST[s], ST[e]) // Place the pivot element at index e
13. If s < e-1
14. Recursively call Quicksort(ST, s, e-1)
15. End if
16. If e+1 < l
17. Recursively call Quicksort(ST, e+1, l)
18. End if
19. Stop

Algorithm for Binary Search
Algorithm Binary_Search (S, n, Key)
Where S is structure array, n is the no of records, and Key is name to be searched.

1. Start
2. Set b = 0 & e = n-1
3. While b ≤ e
4. mid = (b + e) / 2
5. If strcmp(S[mid].Name, key) > 0 // compare two strings
6. e = mid – 1 // left side
7. Else if strcmp(S[mid].Name, key) < 0
8. b = mid + 1 // right side
9. Else
   return mid
10. End While
11. Return -1 // Not Found

Algorithm for Linear search

Algorithm Linear _Search (S, n , Key) key is name to be searched		Where S is structure array , n is the no of records, and
1	Start	
2	For i= 0 to n-1	Set b = 0 & e = n-1
3	If (S[i].SGPA == Key)	//Number Present
4	Display S[i].Rollno and S[i].Name	
5	Break	
6	End if	
7	End for	
8	Return -1 // Not Found	

	Bubble	Insertion Sort	Quick Sort
Best case Time Complexity	O(n)	O(n)	O(n*log n)
Average case Time Complexity	$O(n^2)$	$O(n^2)$	O(n*log n)
worst case Time Complexity	$O(n^2)$	$O(n^2)$	$O(n^2)$
Space Complexity	O(1)	O(1)	O(1)
Stable	Yes	Yes	No
Sorting Method	Internal	Internal	Internal

INPUT:

Enter the 15 records of students , contain Rollno,Name ,CGPA Enter the Sorting techniques (Bubble , Insertion ,Quick)

Enter the Searching techniques (Linear , Binary)

OUTPUT

1. Using Bubble sort, sort the list of students in ascending order
2. Using Insertion sort, sort the list of students alphabetically
3. Using Quick sort , find 10 toppers

4.Using Linear search , search student by SGPA

5. Using Binary search , search student by Name

Oral Questions:

1. List the advantages and disadvantages of Binary Search?
2. Explain the analysis of Binary Search with respect to best case, worst case & Average case?
3. What is In-place & Stable Features of Sorting Algorithms?
4. Explain what you mean by Internal & External Sorting.
5. Explain the advantages and disadvantages of bubble sort
6. Analyze Bubble sort with respect to Time complexity?
7. How to improve the efficiency of Bubble sort if the array gets sorted before the n-1 passes?
8. What is sorting stability?
9. Explain the analysis of Selection Sort and Insertion Sort
10. Give applications of Binary Search?
11. Compare Binary search and linear search with respect to complexity
12. Explain the importance of sorting and searching in computer applications?
13. What is pivot element?
14. What are the ways for choosing the pivot element.?
15. In its worst case QuickSort behaves like:
    1. Bubble sort b) Selection sort c) Insertion sort d) Bin sort?
16. Determine the running time of QuickSort for Sorted input, reverse ordered input, random input, When all the elements are equal.?
17. Suppose we always choose the middle element as the pivot . Does this make it unlikely that QuickSort will require quadratic time?
18. What is the worst-case behavior (number of comparisons) for quick sort?
19. Comment on stability of Quicksort?
20. Explain Big oh ,Omega , Theta Notations

CONCLUSION

Thus we have implemented bubble sort ,insertion sort, quick sort , linear search and binary search technique and analyzed the results for best, worst and average cases.

Assignment No 2:

AIM:

Sort Online Orders by Delivery Time using Merge Sort. Problem Statement: You are given a list of online orders, each with an estimated delivery time in minutes. Write a program to sort these orders using the Merge Sort algorithm so the delivery system can prioritize quicker deliveries first.

Objective

1. To understand the concept of the Divide and Conquer technique used in the Merge Sort algorithm.
2. To implement Merge Sort in C++ to sort online orders based on their estimated delivery time in ascending order.
3. To develop logical thinking and problem-solving skills by dividing a problem into smaller subproblems and combining their solutions.
4. To analyze the time and space complexity of the Merge Sort algorithm.
5. To understand how sorting algorithms can be applied to real-world applications such as online order prioritization and delivery scheduling.

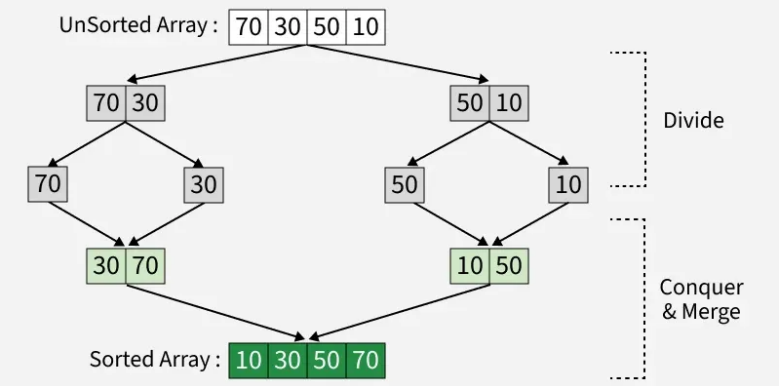
Outcomes: After completing this experiment, students will be able to:

1. Explain the working principle of the Merge Sort algorithm and the Divide and Conquer technique.
2. Write and execute a C++ program to sort online orders by estimated delivery time using Merge Sort.
3. Apply the divide, conquer, and merge steps to sort a given list of elements.
4. Analyze the best-case, average-case, and worst-case time complexity of Merge Sort as (O(n log n)).
5. Identify practical applications of Merge Sort in order management, scheduling, and data processing.

Theory:

Merge Sort:

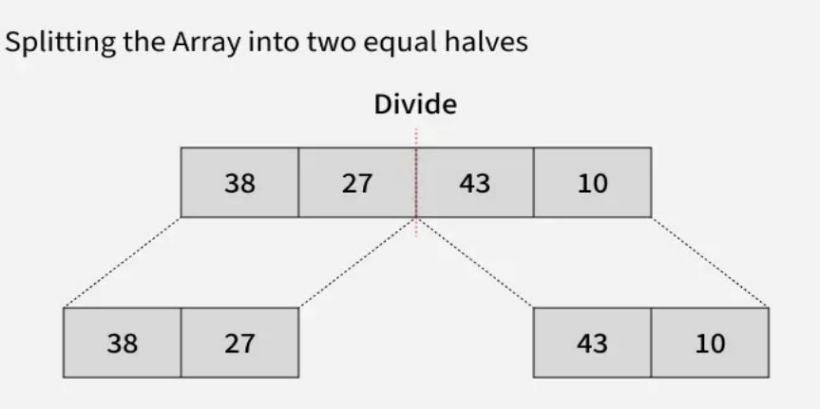
- Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the Divide and Conquer approach. It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.

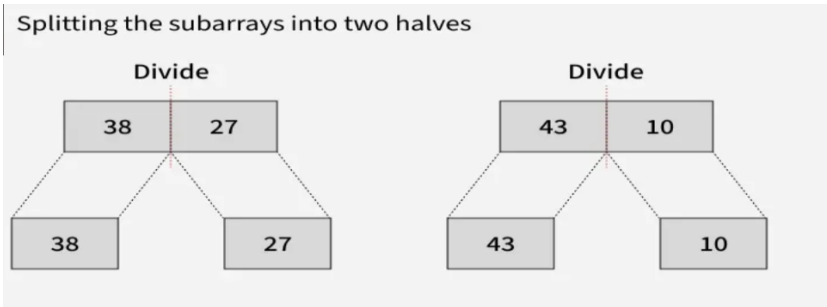


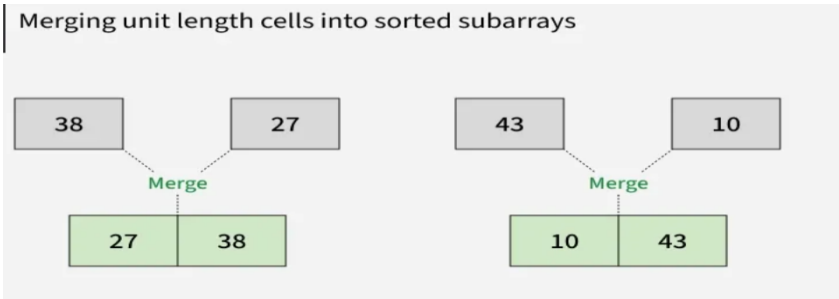
Here's a step-by-step explanation of how merge sort works:

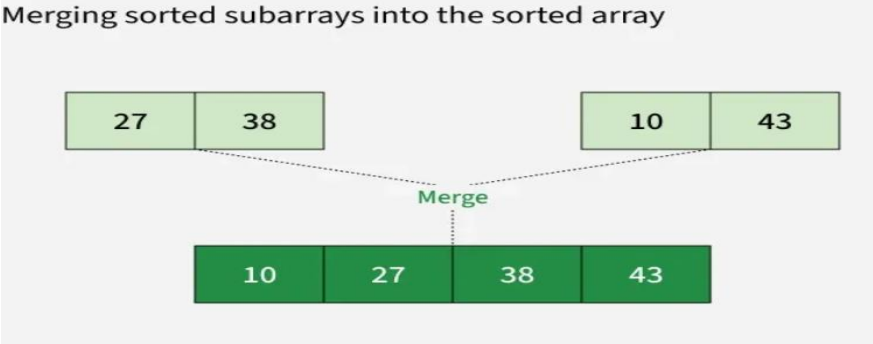
1. Divide: Divide the list or array recursively into two halves until it can no more be divided.
2. Conquer: Each subarray is sorted individually using the merge sort algorithm.
3. Merge: The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

Let's sort the array or list [38, 27, 43, 10] using Merge Sort









Let's look at the working of above example:
Divide:

- [38, 27, 43, 10] is divided into [38, 27] and [43, 10] .
- [38, 27] is divided into [38] and [27] .
- [43, 10] is divided into [43] and [10] .

Conquer:

- [38] is already sorted.
- [27] is already sorted.
- [43] is already sorted.
- [10] is already sorted.

Merge:

- Merge [38] and [27] to get [27, 38] .
- Merge [43] and [10] to get [10,43] .
- Merge [27, 38] and [10,43] to get the final sorted list [10, 27, 38, 43]

Therefore, the sorted list is [10, 27, 38, 43].

Algorithm

Step 1: Start.

Step 2: Read the number of online orders, n.

Step 3: Accept the estimated delivery time of each order.

Step 4: Divide the array into two halves using the middle index.

Step 5: Recursively apply Merge Sort to both halves until each subarray contains only one element.

Step 6: Merge the two sorted subarrays by comparing their elements and placing the smaller element first.

Step 7: Repeat the merging process until the complete array is sorted.

Step 8: Display the sorted delivery times in ascending order.

Step 9: Stop.

Recurrence Relation of Merge Sort

The recurrence relation of merge sort is:
T(n)={Θ(1)if n=12T(n2)+Θ(n)if n>1$T(n)$={Θ(1)2$T(2n)$+Θ(n)if n=1if n>1

- T(n) Represents the total time time taken by the algorithm to sort an array of size n.
- 2T(n/2) represents time taken by the algorithm to recursively sort the two halves of the array. Since each half has n/2 elements, we have two recursive calls with input size as (n/2).
- O(n) represents the time taken to merge the two sorted halves

Complexity Analysis of Merge Sort

Time Complexity:
- Best Case: O(n log n), When the array is already sorted or nearly sorted.
- Average Case: O(n log n), When the array is randomly ordered.
- Worst Case: O(n log n), When the array is sorted in reverse order.

Auxiliary Space: O(n), Additional space is required for the temporary array used during merging.

Applications of Merge Sort:

- Sorting large datasets
- External sorting (when the dataset is too large to fit in memory)
- Used to solve problems like Inversion counting, Count Smaller on Right & Surpasser Count
- Merge Sort and its variations are used in library methods of programming languages. Its variation Time Sort is used in Python, Java Android and Swift. The main reason why it is preferred to sort non-primitive types is stability which is not there in QuickSort. Arrays. Sort in Java uses Quick Sort while Collections sort uses Merge Sort.
- It is a preferred algorithm for sorting Linked lists.
- It can be easily parallelized as we can independently sort sub arrays and then merge.
- The merge function of merge sort to efficiently solve the problems like union and intersection of two sorted arrays.

Advantages and Disadvantages of Merge Sort

Advantages
- Stability: Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.

- Guaranteed worst-case performance: Merge sort has a worst-case time complexity of O(N logN) , which means it performs well even on large datasets.
- Simple to implement: The divide-and-conquer approach is straightforward.
- Naturally Parallel: We independently merge subarrays that makes it suitable for parallel processing.

Disadvantages
- Not an in-place, which means it requires additional memory to store the sorted data. This can be a disadvantage in applications where memory usage is a concern.
- Slower than Quick Sort in general as Quick Sort is more cache friendly because it works in-place.

Conclusion:
The Merge Sort algorithm was successfully implemented in C++ to sort online orders according to their estimated delivery times in ascending order. The experiment demonstrated the Divide and Conquer technique by dividing the array into smaller subarrays, sorting them, and merging them into a sorted list. Merge Sort provides an efficient time complexity of \(O(n\log n)\), making it suitable for sorting large datasets and prioritizing online orders with shorter delivery times.

Viva Question

1. What is Merge Sort?

2. What is the Divide and Conquer technique?

3. How does the Merge Sort algorithm work?

4. What is the best-case time complexity of Merge Sort?

5. What is the worst-case time complexity of Merge Sort?

6. What is the space complexity of Merge Sort?

7. Is Merge Sort a stable sorting algorithm?

8. What is the purpose of the merge() function?

9. What is the purpose of the mergeSort() function?

10. What is the base condition in Merge Sort?

11. Why is Merge Sort suitable for sorting online orders?

12. How is the middle index calculated in the Merge Sort program?

13. What happens if two orders have the same delivery time?

14. Can Merge Sort be used to sort elements in descending order?

15. How is Merge Sort different from Bubble Sort

Assignment No :3

AIM:

Implement stack as an abstract data type using singly linked list and use this ADT for conversion of infix expression to postfix, prefix

Objectives:

1. To understand the concept of Stack ADT.
2. To implement stack operations using a singly linked list.
3. To perform push, pop, and peek operations.
4. To convert infix expressions into postfix expressions.

Theory:

Stack: A stack is a linear data structure that follows the LIFO (Last In, First Out) principle. The element inserted last is removed first.

Abstract Data Type (ADT): An ADT defines the operations that can be performed on a data structure without specifying all implementation details.

Singly Linked List: A linked list consists of nodes. Each node contains data and a pointer to the next node.

Stack operations

- Push: Inserts an element at the top of the stack.
- Pop: Removes the top element from the stack.
- Peek: Returns the top element without removing it.
- IsEmpty: Checks whether the stack is empty.

Types of expressions

Expression	Description	Example
Infix	Operator is between operands	A+B
Postfix	Operator is after operands	AB+
Prefix	Operator is before operands	+AB

Operator precedence

Operator	Precedence
^	Highest
*, /, %	Medium
+, -	Lowest

Parentheses are used to control the order of evaluation. The exponentiation operator ^ is generally right-associative, while the other listed binary operators are left-associative.

4. Algorithm

A. Stack using a singly linked list

1. Create a node containing data and a pointer to the next node.
2. Initialize top to NULL.
3. For push, insert a new node at the beginning of the linked list.
4. For pop, remove the first node and update top.
5. For peek, display the data stored at top.
6. If top == NULL, the stack is empty.

B. Infix to postfix conversion

1. Scan the infix expression from left to right.
2. If the symbol is an operand, add it to the postfix expression.
3. If it is an opening parenthesis (, push it onto the stack.
4. If it is a closing parenthesis), pop operators until (is found.
5. If it is an operator, pop operators of higher precedence, and equal precedence when appropriate for left-associative operators.
6. Push the current operator onto the stack.
7. After scanning the expression, pop all remaining operators.

C. Infix to prefix conversion

1. Reverse the infix expression.
2. Interchange opening and closing parentheses.
3. Convert the modified expression into postfix, handling associativity correctly.
4. Reverse the resulting postfix expression to obtain the prefix expression.

Example

Given infix expression:

(A+B)*C-D

Step	Operation	Stack	Output
1	Push (	(	
2	Add A	(	A
3	Push +	(+	A
4	Add B	(+	AB
5	Pop + and discard (	Empty	AB+
6	Push *	*	AB+

7	Add C	*	AB+C
8	Pop *, push -	-	AB+C*
9	Add D	-	AB+C*D
10	Pop remaining operator	Empty	AB+C*D-

Result:

- Infix: (A+B)*C-D
- Postfix: AB+C*D-
- Prefix: -*+ABCD

Time and Space Complexity:

Operation	Complexity
Push	\(O(1)\)
Pop	\(O(1)\)
Peek	\(O(1)\)
Infix to postfix	\(O(n)\)
Infix to prefix	\(O(n)\)
Auxiliary space for conversion	\(O(n)\)

Here, (n) is the number of characters in the expression.

9. **Learning Outcomes**

After completing this experiment, students will be able to:

1. Understand and implement Stack ADT using a singly linked list.
2. Perform push, pop, peek, and empty-stack operations.
3. Apply stack operations to expression conversion.
4. Convert infix expressions into postfix and prefix forms.
5. Analyze the time and space complexity of stack-based algorithms.

10. Conclusion

Thus, the Stack ADT was implemented using a singly linked list, and the stack was used to convert infix expressions into postfix and prefix expressions. The experiment demonstrates the practical application of stacks in expression processing and operator precedence management.

Viva Questions

1. What is a stack?
2. What is Stack ADT?
3. What is a singly linked list?
4. Why do we use a linked list to implement a stack?
5. What is the LIFO principle?
6. What is the push operation?
7. What is the pop operation?
8. What are stack overflow and stack underflow?
9. What is the difference between infix, postfix, and prefix expressions?
10. Why is a stack used for expression conversion?
11. What is operator precedence?
12. What is associativity?
13. What is the time complexity of push and pop operations in a linked-list stack?
14. What is the time complexity of infix-to-postfix conversion?
15. Convert the infix expression (A+B)*C into postfix and prefix

Experiment No . 4

AIM:

Implement Circular Queue using Array. Perform following operations on it.

a) Insertion (Enqueue)

b) Deletion (Dequeue)

c) Display (Note: Handle queue full condition by considering a fixed size of a queue.)

Implementation of Circular Queue Using Array

A circular queue is a linear data structure that overcomes the limitations of a simple queue. In a normal array implementation, dequeue() can be O(n) or we may waste space. Using a circular array, both enqueue() and dequeue() can be done in O(1).

Declaration using Array:

In this implementation, arr is used to store elements. some variables are maintained:

- arr[] : array to store elements.
- capacity : maximum size of the queue.
- front : index of the front element.
- size : current number of elements in the queue.

Theory:

1) What is Queue?

A queue is a particular kind of collection in which the entities in the collection are kept in order and the principal (or only) operations on the collection are the addition of entities to the rear terminal position and removal of entities from the front terminal position. This makes the queue a First-In-First-Out (FIFO) data structure. In a FIFO data structure, the first element added to the queue will be the first one to be removed. This is equivalent to the requirement that once an element is added, all elements that were added before have to be removed before the new element can be invoked. A queue is an example of a linear data structure.

Queues provide services in computer science, transport, and operations research where various entities such as data, objects, persons, or events are stored and held to be processed later. In these contexts, the queue performs

the function of a buffer. Queues are common in computer programs, where they are implemented as data structures coupled with access routines, as an abstract data structure or in object-oriented languages as classes. Common implementations are circular buffers and linked lists. Queue is a data structure that maintain "First In First Out" (FIFO) order. And can be viewed as people queueing up to buy a ticket. In

programming, queue is usually used as a data structure for BFS (Breadth First Search)

Operations On Circular Queue:

enqueue(x) :
Purpose: Insert an element x at the rear of the circular queue.

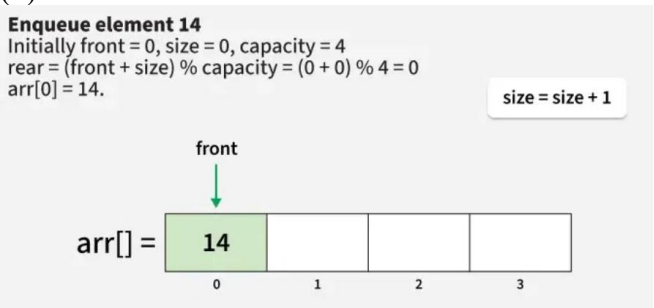
1. Check for full queue: If size == capacity, the queue is full, print message or return.
2. Compute rear index: rear = (front + size) % capacity ensures circular behavior.
3. Insert element: arr[rear] = x.
4. Update size: Increment size by 1.

Time Complexity: O(1)
Space Complexity: O(1) for the array.

dequeue() :
Purpose: Remove and return the front element from the circular queue.

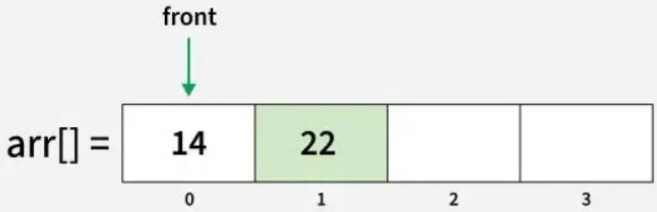
1. Check for empty queue: If size == 0, the queue is empty, print message or return -1.
2. Retrieve front element: res = arr[front].
3. Move front forward: front = (front + 1) % capacity (circular movement).
4. Update size: Decrement size by 1.
5. Return element: Return res.

Time Complexity: O(1)
Space Complexity: O(1)



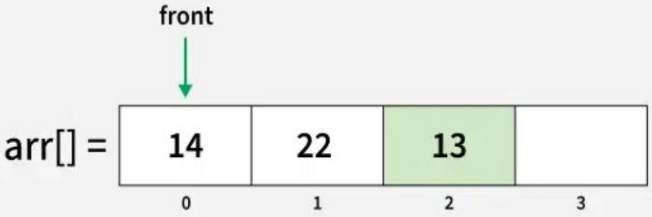
Enqueue element 22
Now front = 0, size = 1, capacity = 4
rear = (front + size) % capacity = (0 + 1) % 4 = 1
arr[1] = 22.

size = size + 1



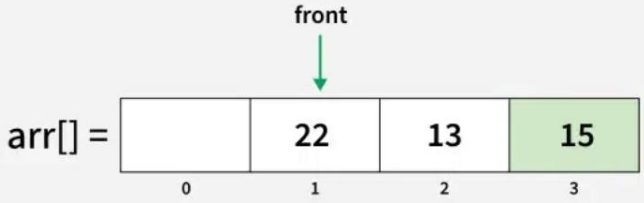
Enqueue element 13
Now front = 0, size = 2, capacity = 4
rear = (front + size) % capacity = (0 + 2) % 4 = 2
arr[2] = 13.

size = size + 1



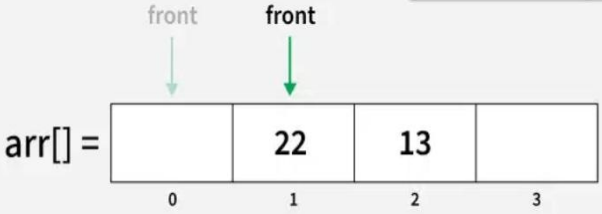
Enqueue element 15
Now front = 1, size = 2, capacity = 4
rear = (front + size) % capacity = (1 + 2) % 4 = 3
arr[3] = 15.

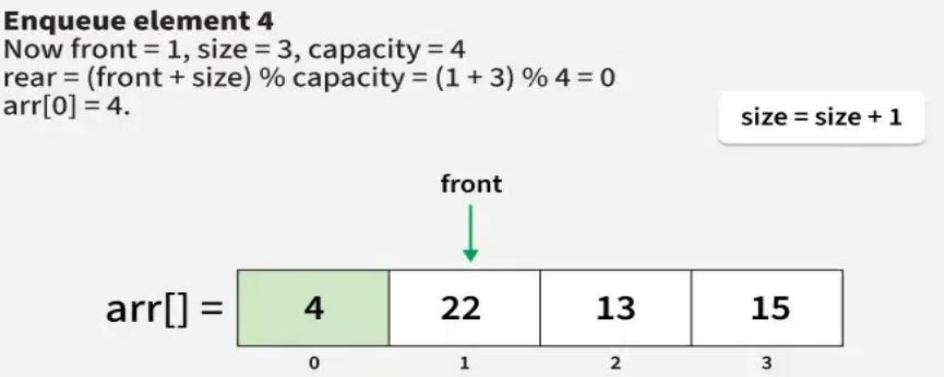
size = size + 1



Dequeue
Now front = 0, size = 3, capacity = 4.

front = (front + 1) % capacity
size = size - 1





What is Circular Queue ? Explain with example.

In a standard queue data structure re-buffering problem occurs for each dequeue operation. To solve this problem by joining the front and rear ends of a queue to make the queue as a circular queue. Circular queue is a linear data structure. It follows FIFO principle.

- In circular queue the last node is connected back to the first node to make a circle.
- Circular linked list fallow the First In First Out principle
- Elements are added at the rear end and the elements are deleted at front end of the queue
- Both the front and the rear pointers points to the beginning of the array.
- It is also called as “Ring buffer”.
- Items can inserted and deleted from a queue in O(1) time.

Circular Queue can be created in three ways they are

- Using single linked list
- Using double linked list
- Using arrays

Using single linked list:

It is an extension for the basic single linked list. In circular linked list Instead of storing a Null value in the last node of a single linked list, store the address of the 1st node (root) forms a circular linked list. Using circular linked list it is possible to directly traverse to the first node after reaching the last node.

Using double linked list

In double linked list the right side pointer points to the next node address or the address of first node and left side pointer points to the previous node address or the address of last node of a list. Hence the above list is known as

circular double linked list.

Using array

In arrays the range of a subscript is 0 to n-1 where n is the maximum size. To make the array as a circular array by making the subscript 0 as the next address of the subscript n-1 by using the formula subscript = (subscript +1) % maximum size. In circular queue the front and rear pointer are updated by using the above formula.

Oral Questions

1. What are the types of data structure?
2. What are the operations can implement on queue?
3. What is circular queue?
4. What is Multiqueue?
5. Applications of stack and Queue as a data structure
6. Write a program to implement Queue using linked list

CONCLUSION

Studied Queue Data Structure using arrays and linked list

Assignment No: 5

AIM:

Implement binary search tree and perform following operations:

a) Insert (Handle insertion of duplicate entry

b) Delete

c) Search

d) Display tree (Traversal)

e) Display - Depth of tree

f) Display - Mirror image

g) Create a copy

h) Display all parent nodes with their child nodes

i) Display leaf nodes j) Display tree level wise

OBJECTIVE:

1. To understand the concept of binary search tree as a data structure.
2. Applications of BST.

OUTCOME :

1. To construct binary search tree.
2. To perform different operations on it.
3. To do traversals on a tree.

THEORY:

1. Definition of binary search tree :

A binary tree in which each internal node x stores an element such that the element stored in the left subtree of x are less than or equal to x and elements stored in the right subtree of x are greater than or equal to x. This is called binary-search-tree property.

2. Illustrate the above operations.

Searching:
Searching a binary tree for a specific value can be a recursive or iterative process. This explanation covers a recursive method.We begin by examining the root node. If the tree is null, the value we are searching for does not exist in the tree. Otherwise, if the value equals the root, the search is successful. If the value is less than the root, search the left subtree. Similarly, if it is greater than the root, search the right subtree. This process is repeated until the value is found or the indicated subtree is null. If the searched value is not found before a

null subtree is reached, then the item must not be present in the tree.

Insertion

Insertion begins as a search would begin; if the root is not equal to the value, we search the left or right subtrees as before. Eventually, we will reach an external node and add the value as its right or left child, depending on the node's value. In other words, we examine the root and recursively insert the new node to the left subtree if the new value is less than the root, or the right subtree if the new value is greater than or equal to the root.

Deletion

There are three possible cases to consider:

- Deleting a leaf (node with no children): Deleting a leaf is easy, as we can simply remove it from the tree.
- Deleting a node with one child: Remove the node and replace it with its child.
- Deleting a node with two children: Call the node to be deleted N. Do not delete N. Instead, choose either its in-order successor node or its in-order predecessor node, R. Replace the value of N with the value of R, then delete R.

As with all binary trees, a node's in-order successor is the left-most child of its right subtree, and a node's in-order predecessor is the right-most child of its left subtree. In either case, this node will have zero or one children. Delete it according to one of the two simpler cases above.

ALGORITHM:

Define structure for Binary Tree (Mnemonics, Left Pointer, Right Pointer)

Insert Node:

Insert(Root, Node)

Root is a variable of type structure ,represent Root of the Tree. Node is a variable of type structure ,represent new Node to insert in a tree.

Step 1: Repeat Steps 2,3 & 4 Until Node do not insert at

appropriate position. Step 2: If Node Data is Less that Root Data & Root Left Tree is NULL

Then insert Node to Left.

Else Move Root to Left

Step 3 : Else If Node Data is Greater that Equal that Root Data & Root Right Tree is NULL Then insert Node to Right.

Else Move Root to Right.

Step 4: Stop.

Search Node:

Search (Root, Mnemonics)

Root is a variable of type structure ,represent Root of the Tree.

Mnemonics is array of character. This function search Mnemonics in a Tree.

Step 1: Repeat Steps 2,3 & 4 Until Mnemonics Not find && Root != NULL Step 2: If Mnemonics Equal to Root Data

Then print message Mnemonics present.

Step 3 : Else If Mnemonics Greater that Equal that Root Data Then Move Root to Right

Step 4 : Else Move Root to Left.

Step 5: Stop.

Delete Node:

Dsearch(Root, Mnemonics)

Root is a variable of type structure ,represent Root of the Tree. Mnemonics is array of character. Stack is an pointer array of type structure. PTree(Parent of Searched Node),Tree(Node to be deleted), RTree(Pointg to Right Tree),Temp are pointer variable of type structure;

Step 1: Search Mnemonics in a Binary Tree
Step 2: If Root == NULL Then Tree is NULL
Step 3: Else // Delete Leaf Node
If Tree->Left == NULL && Tree->Right == NULL Then
a) If Root == Tree Then Root = NULL;
b) If Tree is a Right Child PTree->Right = NULL; Else PTree->Left = NULL;
Step 4: Else // Delete Node with Left and Right children
If Tree->Left != NULL && Tree->Right != NULL Then
a) RTree = Temp = Tree->Right;
b) Do steps i && ii while Temp->Left != NULL
i) RTree = Temp;
ii) Temp = Temp->Left;
c) RTree->Left = Temp->Right;
d) If Root == Tree // Delete Root Node
Root = Temp;
e) If Tree is a Right Child PTree->Right = Temp; Else PTree->Left = Temp;
f) Temp->Left = Tree->Left;
g) If RTree != Temp
Then Temp->Right = Tree->Right;
Step 5: Else // With Right child
If Tree->Right != NULL Then
a) If Root == Tree Root = Root->Right;
b) If Tree is a Right Child PTree->Right = Tree->Right;
Else PTree->Left = Tree->Left;
Step 6: Else // With Left child
If Tree->Left != NULL Then
a) If Root == Tree Root = Root->Left;
b) If Tree is a Right Child PTree->Right = Tree->Left; Else PTree->Left = Tree->Left;
Step 7: Stop.

Depth First Search:

Root is a variable of type structure ,represent Root of the Tree.

Stack is an pointer array of type structure. Top variable of type

integer. DFS(Root)

Step 1: Repeat Steps 2,3,4,5,6

Until Stack is Empty Step 2: print

Root Data

Step 3: If Root->Right != NULL//Root Has a

Right SubTree Then Stack[Top++] =

Tree->Right;//Push Right Tree into

Stack Step 4: Root = Root ->Left;//Move to Left

Step 5: If Root == NULL

Step 6: Root = Stack[--Top];//Pop

Node from Staacl Step 7: Stop.

Breath First Searc(Levelwise Display):

Root is a variable of type structure ,represent Root of the Tree.

Queue is an pointer array of type structure. Front & Rear

variable of type integer. BFS(Root)

Step 1:If Root == NULL Then Empty Tree;

Step 2: Else Queue[0] = Root;// insert Root of

the Tree in a Queue Step 3: Repeat Steps 4,5,6

& 7 Until Queue is Empty

Step 4: Tree=Queue[Front++]; //Remove

Node From Queue Step 5: print Root Data

Step 6: If Root->Left != NULL

Then Queue[++Rear] = Tree->Left;//insert Left

Subtree in a Queue Step 7: If Root->Right != NULL

Then Queue[++Rear] = Root->Right; //insert Left Subtree in a Queue
Else if Root->Right == NULL And Root->Left == NULL
Leaf++;//Number of Leaf Nodes

Step 8: Stop.

Mirror Image:

Root is a variable of type structure ,represent Root of the Tree.

Queue is an pointer array of type structure. Front & Rear variable of type

integer. Mirror(Root) Step 1: Queue[0]=Root;//Insert Root Node in a Queue

Step 2: Repeat Steps 3,4,5,6,7 & 8 Until

Queue is Empty Step 3: Root =

Queue[Front++];

Step 4:
Temp1 =
Root->Left;
Step 5: Root-
>Left = Root-
>Right; Step 6:
Root->Right =
Temp1; Step 7: If
Root->Left !=
NULL
Then Queue[Rear++] = Tree-
>Left;//insert Left SubTree Step 8: If Root-
>Right != NULL
Then Queue[Rear++] = Root-
>Right;//insert Right SubTree Step 9: Stop.

Depth of tree:

Root is a variable of type structure ,represent Root of the Tree.
Stack is an pointer array of type structure. Top variable of type integer.

DFS(Root)

Step 1: Repeat Steps 2,3,4,5,6
Until Stack is Empty Step 2: print
Root Data
Step 3: If Root->Right != NULL//Root Has a
Right SubTree Then Stack[Top++] =
Tree->Right;//Push Right Tree into
Stack Step 4: Root = Root -
>Left;//Move to Left Step 5: If
Root == NULL
Step 6: Root = Stack[--Top];//Pop
Node from Stack Step 7: Stop.

Display leaf nodes:

Root is a variable of type structure ,represent Root of the Tree.

Queue is an pointer array of type structure. Front & Rear

variable of type integer. **BFS(Root)**

Step 1:If Root == NULL Then Empty Tree;

Step 2: Else Queue[0] = Root;// insert Root of

the Tree in a Queue Step 3: Repeat Steps 4,5,6

& 7 Until Queue is Empty

Step 4: Tree=Queue[Front++]; //Remove

Node From Queue Step 5: print Root Data

Step 6: If Root->Left != NULL

Then Queue[++Rear] = Tree->Left;//insert Left

Subtree in a Queue Step 7: If Root->Right != NULL

Then Queue[++Rear] = Root->Right; //insert Left

Subtree in a Queue Else if Root->Right == NULL

And Root->Left == NULL

Leaf++;//Number of Leaf Nodes

Step 8: Stop.

INPUT:

Accept the nodes from the user like: add, mult, div, sub or(11, 22 ,33 ,44)

OUTPUT:

Display result of each operation with error checking.

Oral Questions

1. What is Binary search tree?
2. What are the members of structure of tree & what is the size of structure?
3. What are rules to construct binary search tree?
4. How general tree is converted into binary tree?
5. What is binary threaded tree?
6. What is Binary tree, Binary search tree, Expression tree & General tree?
7. What is preorder, postorder, inorder traversal?
8. Difference between recursive & Nonrecursive traversal?

9. What are rules to construct binary search tree?
10. What are rules to construct binary tree?
11. How binary tree is constructed from its traversals?

CONCLUSION

Studied the Binary tree, Binary search tree and operations on it

Assignment No: 6

AIM:

Design and implement the Heap Sort algorithm to efficiently sort an array of integers in ascending order. The implementation should be optimized for time and space complexity and should clearly demonstrate the working principles of heap data structures (min-heap or max heap as applicable)

OUTCOME:

1. Understand the properties of Heap data structure.
2. Implement Heap sort.
3. Analysis of heap sort

THEORY:

What is a heap sort?

Heapsort is a comparison-based sorting algorithm to create a sorted array (or list), and is part of the selection sort family. Although somewhat slower in practice on most machines than a well-implemented quicksort, it has the advantage of a more favorable worst-case O(n log n) runtime. Heapsort is an in-place algorithm, but it is not a stable sort.

The heapsort algorithm can be divided into two parts. In the first step, a heap is built out of the data. In the second step, a sorted array is created by repeatedly removing the largest element from the heap, and inserting it into the array. The heap is reconstructed after each removal. Once all objects have been removed from the heap, we have a sorted array. The direction of the sorted elements can be varied by choosing a min-heap or max-heap in step one. Heapsort can be performed in place. The array can be split into two parts, the sorted array and the heap.

The (Binary) heap data structure is an array object that can be viewed as a nearly complete binary tree. A binary tree with n nodes and depth k is complete iff its nodes correspond to the nodes numbered from 1 to n in the full binary tree of depth k

Heap properties

There are two kind of binary heaps: max-heaps and min-heaps.

In a max-heap, the max-heap property is that for every node I other than

the root, $A[PARENT(i)] \geq A[i]$.

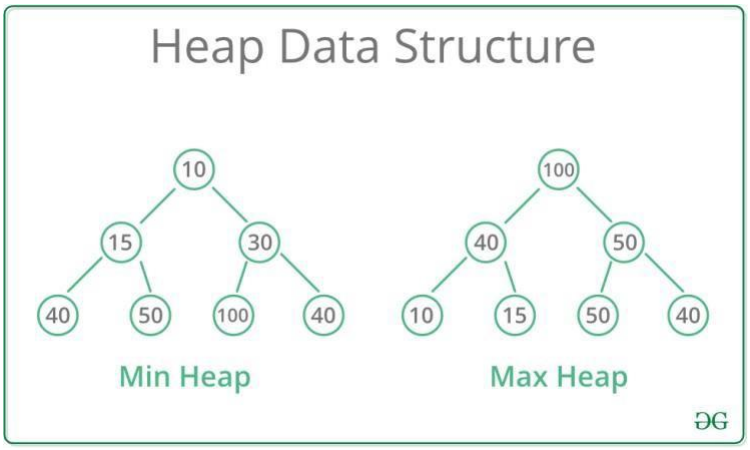
the largest element in a max-heap is stored at the root

the subtree rooted at a node contains values no larger than that contained at the node itself In a min-heap, the min-heap property is that for every node *I* other than the root,

$A[PARENT(i)] \leq A[i]$.

the smallest element in a min-heap is at the root

the subtree rooted at a node contains values no smaller than that contained at the node itself



Comparison with other sorts

Heapsort primarily competes with quicksort, another very efficient general purpose nearly-in-place comparison-based sort algorithm. Quicksort is typically somewhat faster due to some factors, but the worst-case running time for quicksort is $O(n^2)$, which is unacceptable for large data sets and can be deliberately triggered given enough knowledge of the implementation, creating a security risk. See quicksort for a detailed discussion of this problem and possible solutions. Thus, because of the $O(n \log n)$ upper bound on heapsort's running time and constant upper bound on its auxiliary storage, embedded systems with real-time constraints or systems concerned with security often use heapsort.

Heapsort also competes with merge sort, which has the same time bounds. Merge sort requires $\Omega(n)$ auxiliary space, but heapsort requires only a constant amount. Heapsort typically runs faster in practice on machines with small or slow data caches. On the other hand, merge sort has several advantages over heapsort:

- Merge sort on arrays has considerably better data cache performance, often outperforming heapsort on modern desktop computers because merge sort frequently accesses contiguous memory locations (good locality of reference); heapsort references are spread throughout the heap.
- Heapsort is not a stable sort; merge sort is stable.
- Merge sort parallelizes well and can achieve close to linear speedup with a trivial implementation; heapsort is not an obvious candidate for a parallel algorithm.
- Merge sort can be adapted to operate on linked lists with O(1) extra space.Heapsort can be adapted to operate on doubly linked lists with only O(1) extra space overhead.
- Merge sort is used in external sorting; heapsort is not. Locality of reference is the issue.

Introsort is an interesting alternative to heapsort that combines quicksort and heapsort to retain advantages of both: worst case speed of heapsort and average speed of quicksort.

The heapify function can be thought of as building a heap from the bottom up, successively shifting downward to establish the heap property. An alternative version (shown below) that builds the heap top-down and sifts upward may be conceptually simpler to grasp. This "siftUp" version can be visualized as starting with an empty heap and successively inserting elements, whereas the "siftDown" version given above treats the entire input array as a full, "broken" heap and "repairs" it starting from the last non-trivial sub-heap (that is, the last parent node).

Difference in time complexity between the "siftDown" version and the "siftUp" version.

Also, the "siftDown" version of heapify has $O(n)$ time complexity, while the "siftUp" version given below has $O(n \log n)$ time complexity due to its equivalence with inserting each element, one at a time, into an empty heap.[8] This may seem counter-intuitive since, at a glance, it is apparent that the former only makes half as many calls to its logarithmic-time sifting

function as the latter; i.e., they seem to differ only by a constant factor, which never has an impact on asymptotic analysis.

To grasp the intuition behind this difference in complexity, note that the number of swaps that may occur during any one siftUp call *increases* with the depth of the node on which the call is made. The crux is that there are many (exponentially many) more "deep" nodes than there are "shallow" nodes in a heap, so that siftUp may have its full logarithmic running-time on the approximately linear number of calls made on the nodes at or near the "bottom" of the heap. On the other hand, the number of swaps that may occur during any one siftDown call *decreases* as the depth of the node on which the call is made increases. Thus, when the "siftDown" heapify begins and is calling siftDown on the bottom and most numerous node-layers, each sifting call will incur, at most, a number of swaps equal to the "height" (from the bottom of the heap) of the node on which the sifting call is made. In other words, about half the calls to siftDown will have at most only one swap, then about a quarter of the calls will have at most two swaps, etc.

ALGORITHM:

Heapsort(*a*)

1. Build-max-heap(*a*)
2. for *i* ← *length*[*a*] downto 2
3. do exchange a[1] a[*i*]
4. *heap-size*[a] ← *heap-size*[a] −1
5. max-heapify(*a*, 1)
6. step 4: stop.

heapify(A,i)

1. if (i ≠ l)
2. exchange A[i] with A[l]
3. Heapify(A, l)

Build-max-heap(A)

1. for i = n/2 downto 1

2. Heapify(A, i)

INPUT:

Sequence of elements will be passed as input.

OUTPUT:

Sorted list of elements.

INSTRUCTIONS:

Use of user-defined functions for every operation in mandatory. Proper documentation of the program is a must.

ORAL QUSESTIONS :

1. What is max heap?
2. What is min heap?
3. Explain the best case ,worst case and average case complexity of heap What are the advantages & Disadvantage of Heap sort ?

CONCLUSION

Implemented the MAX HEAP Data structures and studied the complexity

Assignment No: 7

AIM:

Represent a graph of your college campus using adjacency list /adjacency matrix. Nodes should represent the various departments/institutes and links should represent the distance between them. Find minimum spanning tree Using Prims & Kruskal's algorithm.

OBJECTIVE:

1. Learn the concepts of graph as a data structure and their applications in everyday life.
2. Understand graph representation (adjacency matrix, adjacency list, adjacency multi-list).

OUTCOME:

1. Understand non-linear data structure – graph.
2. Represent a graph using adjacency list / adjacency matrix.
3. Implement Prim's and Kruskal's algorithms.
4. Identify applications of Minimum Spanning Trees.
5. Analyze the time and space complexity.

THEORY:

1. What is a graph? Various terminologies and its applications.

A graph G = (V, E) consists of a finite set V of vertices (nodes) and a finite set E of edges, where every edge joins a pair of vertices. If every edge has a number (weight) attached to it, such as a distance or a cost, the graph is called a weighted graph.

- Loop: an edge that connects a vertex to itself.
- Degree of a vertex: the number of edges incident on the vertex.
- Directed graph (digraph): every edge has a direction, i.e. it is an ordered pair (u, v). In an undirected graph an edge is an unordered pair {u, v}.
- Path: a sequence of distinct vertices a1, a2, …, an such that consecutive vertices are joined by an edge.
- Circuit / Cycle: a path that returns to its starting vertex.
- Connected graph: there is a path between every pair of vertices.
- Tree: a connected graph with no cycles. A tree with n vertices has exactly n – 1 edges.
- Applications: road / rail / airline networks, computer and telephone networks, social networks, circuit design, scheduling, web-page linking, GPS navigation.

2. Different representations of a graph.

(a) Adjacency matrix: a V × V array G[i][j]. For an unweighted graph the entry is 1 if an edge exists and 0 otherwise; for a weighted graph the entry stores the weight (□ / INF when there is no edge). For an undirected graph the matrix is symmetric. Space required is $O(V^2)$.

(b) Adjacency list: an array of linked lists, one list per vertex, each list holding the neighbours of that vertex (with the edge weight). Space required is O(V + E), so it is better for sparse graphs.

(c) Adjacency multi-list: every edge is stored only once in a node that is shared by the lists of both its end vertices; useful for marking edges as visited in undirected graphs.

3. Campus graph used in this experiment.

The campus is modelled with 7 nodes (buildings) and 10 paths. Edge weights are walking distances in metres.

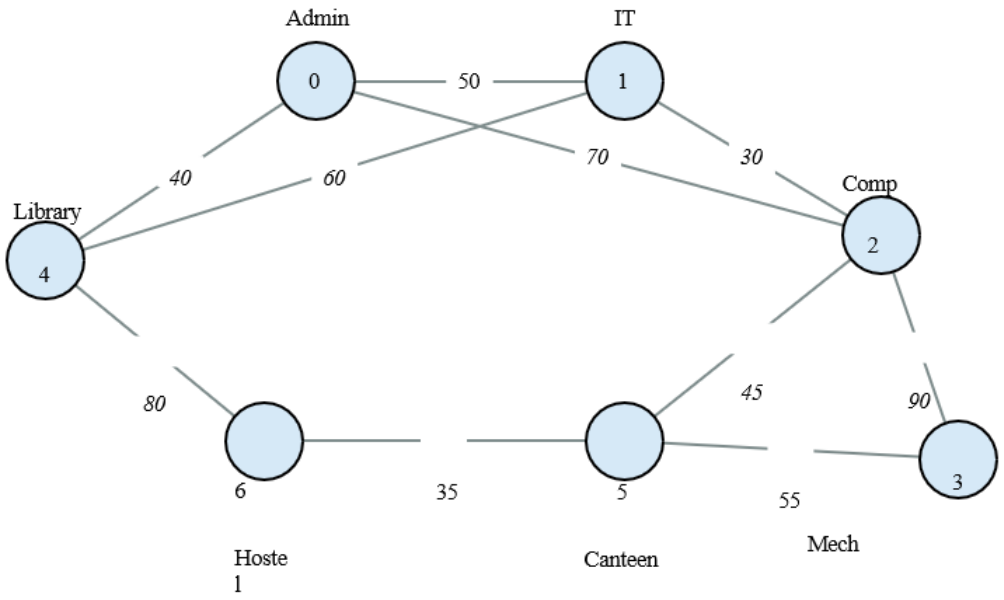


Figure 1: Campus graph (node number inside circle, distance on the edge).

Adjacency matrix (□ = no direct path)

	Admin	IT	Comp	Mech	Library	Canteen	Hostel
Admin	INF	50	70	INF	40	INF	INF
IT	50	INF	30	INF	60	INF	INF
Comp	70	30	INF	90	INF	45	INF
Mech	INF	INF	90	INF	INF	55	INF
Library	40	60	INF	INF	INF	INF	80
Canteen	INF	INF	45	55	INF	INF	35
Hostel	INF	INF	INF	INF	80	35	INF

Adjacency list

Node	Neighbours (distance)
Admin (0)	IT(50) □ Comp(70) □ Library(40)
IT (1)	Admin(50) □ Comp(30) □ Library(60)
Comp (2)	Admin(70) □ IT(30) □ Mech(90) □ Canteen(45)
Mech (3)	Comp(90) □ Canteen(55)
Library (4)	Admin(40) □ IT(60) □ Hostel(80)
Canteen (5)	Comp(45) □ Mech(55) □ Hostel(35)
Hostel (6)	Library(80) □ Canteen(35)

4. Explain Prim's and Kruskal's algorithm.

A spanning tree of a connected undirected graph G is a subgraph that is a tree and contains every

vertex of
G. A Minimum Spanning Tree (MST) is the spanning tree whose total edge weight is the smallest among all spanning trees. Both algorithms below are greedy: they make the locally cheapest choice at every step and still reach a globally optimal tree.

Prim's algorithm grows a single tree. It starts from any vertex and, at each step, adds the minimum-weight edge that connects a vertex already in the tree to a vertex outside the tree. It requires the graph to be connected.

Kruskal's algorithm treats every vertex as a separate tree (forest). It sorts all edges by weight and picks them in increasing order, rejecting any edge that would form a cycle, until n – 1 edges have been chosen. It can also give a minimum spanning forest for a disconnected graph.

ALGORITHM:

Prim's Algorithm:

Step 1: Select any node to be the first node of T (the tree).
Step 2: Consider the arcs which connect nodes in T to nodes outside T. Pick the one with minimum weight. Add this arc and the extra node to T. (If there are two or more arcs of minimum weight, choose any one of them.)
Step 3: Repeat Step 2 until T contains every node of the graph.

Prim(cost[][], n, start)

```
1. for i = 0 to n-1 : dist[i] = INF, from[i] = -1, inMST[i] = false
2. dist[start] = 0
3. repeat n times
4.     u = vertex not in MST having minimum dist[u]
5.     inMST[u] = true ; if from[u] != -1 print edge (from[u], u, dist[u])
6.     for each vertex v not in MST
7.         if cost[u][v] < dist[v] : dist[v] = cost[u][v], from[v] = u
8. total = sum of printed edge weights
9. Stop
```

Kruskal's Algorithm:

Step 1: Choose the arc of least weight.
Step 2: Choose from those arcs remaining the arc of least weight which does not form a cycle with already chosen arcs. (If there are several such arcs, choose one arbitrarily.)
Step 3: Repeat Step 2 until n – 1 arcs have been chosen.

Kruskal(edges[], n, e)

```
1. for i = 0 to n-1 : parent[i] = i                    // each vertex is its own set
2. sort edges[0..e-1] in ascending order of weight
3. count = 0, total = 0
4. for each edge (u, v, w) in sorted order, while count < n-1
5.     a = Find(u), b = Find(v)
6.     if a != b                                       // no cycle formed
7.         parent[a] = b                               // Union
8.         print edge (u, v, w), total = total + w, count = count + 1
9. Stop
```

Working on the campus graph

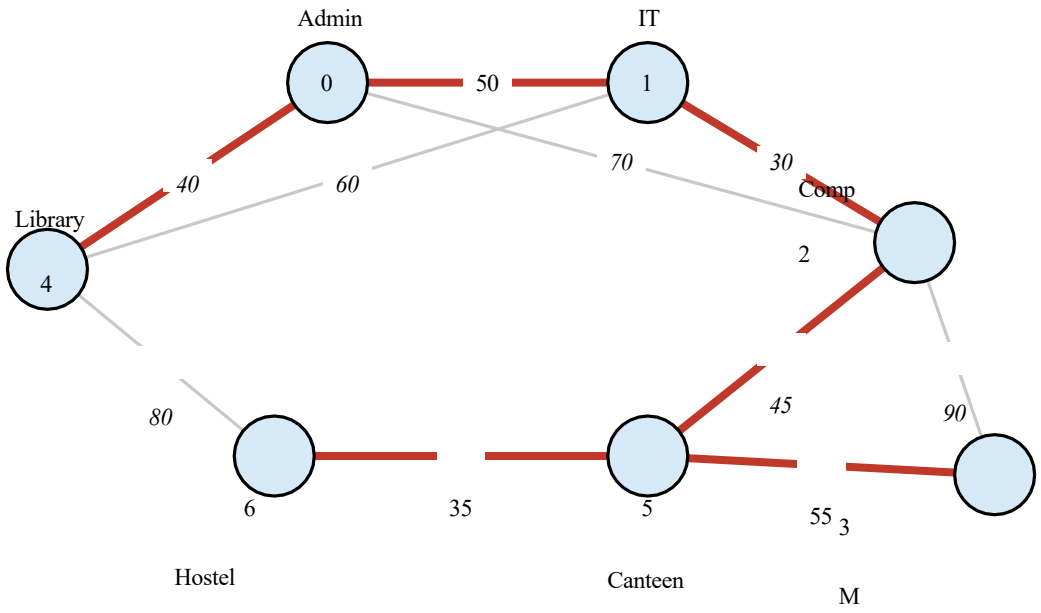
(a) Kruskal's – edges in sorted order

#	Edge	Weight	Cycle?	Decision
1	IT – Comp	30	No	Accept
2	Canteen – Hostel	35	No	Accept
3	Admin – Library	40	No	Accept
4	Comp – Canteen	45	No	Accept
5	Admin – IT	50	No	Accept
6	Mech – Canteen	55	No	Accept (6 edges = n–1, stop)
7	IT – Library	60	Yes	Reject
8	Admin – Comp	70	Yes	Reject
9	Library – Hostel	80	Yes	Reject
10	Comp – Mech	90	Yes	Reject

(b) Prim's – starting at Admin

Step	Vertex added	Edge used	Weight	Tree so far
1	Admin	–	0	{Admin}
2	Library	Admin – Library	40	{Admin, Library}
3	IT	Admin – IT	50	{Admin, Library, IT}
4	Comp	IT – Comp	30	{…, Comp}
5	Canteen	Comp – Canteen	45	{…, Canteen}
6	Hostel	Canteen – Hostel	35	{…, Hostel}
7	Mech	Canteen – Mech	55	All 7 vertices

Both algorithms give the same MST of total weight 40 + 50 + 30 + 45 + 35 + 55 = 255 m.



Oral Questions

1. What is graph?
2. Application of Prim's & Kruskal's algorithm.
3. What are the traversal techniques?
4. What are the graph representation techniques?
5. What is adjacency Matrix?
6. What is adjacency list?
7. What is adjacency Multi-list?

CONCLUSION

Studied the MST using Graph

Assignment No 8

AIM:

Implement a simple telephone directory using a hash table data structure. The goal is to quickly insert, search for, and delete client telephone numbers.

Objective

- To understand the concept of hashing.
- To implement a hash table using an array.
- To perform insertion, searching, and deletion operations.
- To handle collisions using linear probing.

Algorithm

Step 1: Initialize a hash table of fixed size.

Step 2: Calculate the hash index using the hash function:

$$\text{index} = \text{telephone number} \bmod \text{table size}$$

Step 3: Insert the client's name and telephone number at the calculated index. If the location is occupied, use linear probing to find an available slot.

Step 4: Search for a client using their telephone number and display the corresponding name if found.

Step 5: Delete the client's record by searching for the telephone number and marking the slot as deleted.

Step 6: Display all occupied records in the hash table.

Step 7: Stop.

A. Introduction to Hashing

Hashing is a technique used in data structures to store and retrieve data efficiently. It uses a special function called a hash function to convert a key into an index of an array, where the corresponding record is stored.

In a telephone directory, each client has a name and a telephone number. The telephone number can be used as the key to identify the client's record.

Instead of searching through every record sequentially, hashing calculates the location where the record should be stored. This makes insertion and searching faster on average.

For example, consider the following telephone directory:

Client name	Telephone number
Rahul Patil	9876543210
Priya Sharma	9123456780
Amit Shah	9988776655

The telephone number is used as the key, and the client's name and telephone number are stored as a record in the hash table.

B. What Is a Hash Table?

A hash table is a data structure that stores records in an array using a hash function. Each position in the array is called a slot or bucket.

A hash table generally consists of the following components:

- Key: The value used to identify a record, such as a telephone number.
- Value or record: The information associated with the key, such as the client's name and telephone number.
- Hash function: A function that calculates the index at which a record should be stored.
- Table: An array that contains the records.
- Collision resolution technique: A method used when two keys map to the same index.

A hash table can provide average-case \(O(1)\) insertion, searching, and deletion when the hash function distributes keys well and the table is not too full.

C. Hash Function

A hash function converts a key into a valid index in the hash table.

A simple hash function is:

\[h(k)=k \bmod m \]

Where:

- \(h(k)\) = calculated hash index
- \(k\) = numeric key
- \(m\) = size of the hash table
- \(\bmod\) = modulo operation, which returns the remainder

For a telephone number represented as digits, the program can calculate the index by processing the digits and keeping the running result within the table size.

For example, consider a table of size 10 and the telephone number 9876543210.

Using the direct modulo formula:

$$ h(k)=9876543210 \bmod 10=0 $$

Therefore, the initial index is 0.

In the provided C++ program, the hash function processes each digit and keeps the result modulo 10. This produces the same final index as the formula above.

D. Collision in Hashing

A collision occurs when two different keys produce the same hash index.

For example, in a hash table of size 10:

$$ h(25)=25 \bmod 10=5 $$
$$ h(35)=35 \bmod 10=5 $$

Both keys map to index 5. Since only one record can occupy that slot in this implementation, a collision-resolution technique is required.

E. Collision Resolution Using Linear Probing

Linear probing is an open-addressing technique used to resolve collisions. When the calculated position is occupied, the program checks the next position in the table until it finds an available slot.

The formula is:

$$ h_i(k)=(h(k)+i)\bmod m $$

Where:

- $h(k)$ is the original hash index.
- i is the probing attempt, starting from 0.
- m is the table size.

Example with a table of size 10:

Key	Initial index	Final index
25	5	5
35	5	6
45	5	7

The first key occupies index 5. The second key encounters a collision and is placed at index 6. The third key encounters occupied slots 5 and 6 and is placed at index 7.

This process is called linear probing because the program checks successive positions in the table.

F. Operations Performed in the Telephone Directory

1. Insertion

The program accepts the client's name and telephone number, calculates the hash index, and checks whether the location is available. If a collision occurs, linear probing is used to find another slot. Duplicate telephone numbers are rejected.

2. Searching

The program accepts a telephone number and calculates its hash index. It checks the calculated position and follows the same linear probing sequence until the record is found or the search can be stopped.

3. Deletion

The program searches for the specified telephone number. If found, its status is changed to Deleted. The slot is not treated as completely empty because other records may depend on the same probing sequence.

4. Display

The program scans the hash table and displays every occupied slot, showing each client's name and telephone number.

G. Why Is a Deleted Status Required?

In linear probing, simply marking a deleted record as empty can cause searching to stop too early.

Consider the following arrangement:

Index	Key	Status
5	25	Occupied
6	35	Deleted
7	45	Occupied

All three keys originally hash to index 5.

When searching for key 45, the program must continue beyond index 6 to find it at index 7. Therefore, the implementation uses three status values:

- 0 — Empty: the slot has never been occupied.
- 1 — Occupied: the slot contains a valid record.
- 2 — Deleted: the record has been removed, but the probing sequence must be preserved.

This technique is called a tombstone or deleted marker.

H. Hash Table Overflow

Hash table overflow occurs when no available slot remains for a new record.

In the given program, the table has a fixed size of 10. When all slots are occupied and no reusable deleted slot exists, insertion displays the message Directory is full!.

A larger table or a dynamically resized hash table can be used when the number of clients increases.

Time Complexity

Operation	Average case	Worst case
Insertion	\(O(1)\)	\(O(n)\)
Searching	\(O(1)\)	\(O(n)\)
Deletion	\(O(1)\)	\(O(n)\)
Display	\(O(n)\)	\(O(n)\)

Here, (n) represents the hash table size.

Advantages of Hashing

1. Fast searching compared with sequential searching in many practical cases.
2. Efficient insertion and deletion when the table is not heavily loaded.
3. Direct access to records through calculated indices.
4. Suitable for applications involving frequent lookup operations.
5. Easy implementation using arrays and hash functions.
6. Useful for managing telephone directories, student records, employee details, and inventory data.

J. Applications of Hash Tables

Hash tables are widely used in:

- Telephone directories and contact management systems.
- Student information management systems.
- Employee record databases.
- Dictionaries and symbol tables in compilers.
- Caching systems.
- Database indexing and lookup operations.
- Duplicate detection and membership checking

Conclusion

Hashing is an effective technique for storing and retrieving records using a calculated index. By implementing a telephone directory, the concepts of hash functions, collisions, linear probing, and deleted markers were demonstrated. The experiment shows that hash tables provide efficient average-case performance for insertion, searching, and deletion, making them suitable for directory and record-management applications.

Assignment No 9

AIM:
Implement File Handling Operations: Create, Read, Write and Delete

OBJECTIVE:
To understand the implementation of Sequential files.

OUTCOME:

1. Understand file as a permanent data structure.
2. Implement a file and preform various operations on it.
3. To implement the Database application using file data structure

THEORY:

1. **What is file?**

   A file is a collection of data stored in one unit, or resource for storing information identified by a filename, which is available to a computer program and is usually based on some kind of durable storage. A file is durable in the sense that it remains available for programs to use after the current program has finished. It can be a document, picture, audio or video stream, data library, application, or other collection of data

2. **What is a Sequential file and different operations on it?**

   Consider example of a tape where the songs are stored sequentially . whenever we play any song from it , we read it sequentially. Storing data sequentially is the simplest form of file. It is the most common type of file.

   Sequential file is organized by writing the records in the serial order. Thus the first record in the file is the oldest entry & last record corresponds to most recent entry. The record contained in sequential file may be either of fixed or of varying size.

Operations on sequential file:

1) Open: It sets the file pointer to immediately before the first records and allow to access the file.

2) Read: Set the file pointer to the next record in the file and return that record to the user. If there is no record present, then an 'end of file' condition is set.
3) Write/Update: Write backs the current record in the same location where it was read from, with entries modified. The file pointer is unchanged.
4) Append: Position the file pointer to the next of last record and there copies the new record.
5) Close: Terminate the access to the file.
6) EOF: Returns a value of true if the file pointer is beyond the last record of the File, otherwise it is false.

2. Advantages/Disadvantages of Sequential files.
Advantages:
1) Simplest organization
2) Minimum overload on disk
3) Most versatile format
4) Allow back spanning
Disadvantages:
1) To get a particular record, it must access all records before it.
2) No random access by key.

ALGORITHM:
Declare Data structure for given Data(Employee/Student)

Insert:
Insert(File Name)
Step 1: Open File in an append mode.
Step 2: Accept Data.
Step 3: Write Data in a Database File.
Step 4: Close File.
Step 5: Stop.

Search:
Search(File Name, KeyValue)

Step 1: Open File in read mode.

Step 2: Read Data from Database File.

Step 3: If Database Key == KeyValue
Then Record Present, Display Record, Exit.

Step 4: Do Step 2 & 3 till end of file.

Step 5: Close File.

Step 6: Stop.

Delete:
Delete(File Name, KeyValue)

Step 1: Open File in read & write mode.

Step 2: Read Data from Database File.

Step 3: If Database Key == KeyValue
Then Record Present, Seek File pointer at Record Location,
Set Status to -1, Write Record in a File, Exit.

Step 4: Do Step 2 & 3 till end of file.

Step 5: Close File.

Step 6: Stop.

Modify:
Modify(File Name, KeyValue)
Step 1: Open File in read & write mode.

Step 2: Read Data from Database File.

Step 3: If Database Key == KeyValue
Then Record Present, Seek File pointer at Record Location,
Accept New Data, Write Record in a File, Exit.

Step 4: Do Step 2 & 3 till end of file.

Step 5: Close File.
Step 6: Stop.

INPUT:
Enter the students details in file and perform all operations

OUTPUT:
Display result of each operation

ORAL QUESTIONS
1. What is file? What type of data stored in file?
2. What are the different file opening modes?
3. What are different file accessing methods?
4. Compare sequential, index sequential & random access file.

CONCLUSION
Learned different file organizations.